Report 5

Database Design and Module Integration

Course	Com6064 — Software Engineering
Project	QR-Integrated University Room Reservation System
Author	Ibrahim Mutlu (2200005270)
Module	Database & Integration
Date	April 2026
Status	Final

1. Introduction

This report documents the database design and integration work performed for the QR-Integrated University Room Reservation System. While the backend services and the QR generation algorithm were authored by another team member, this document focuses specifically on the data-tier responsibilities: schema design, persistence configuration, and the integration glue that allows the frontend, backend, and database modules to operate as a single coherent system.

The work covered here corresponds to the "Database" and "System Integration" deliverables in the project plan. It assumes familiarity with Reports 1–4, particularly Report 2 (conflict control algorithms) and Report 4 (backend implementation).

2. Purpose of Database Integration

The principal objective of the database integration was to replace the transient in-memory store used during early prototyping with a durable, transactionally consistent PostgreSQL database, and to ensure that the four core entities of the domain — **User**, **Room**, **Reservation**, and **QR** — are mapped consistently between the C# domain layer and their underlying tables.

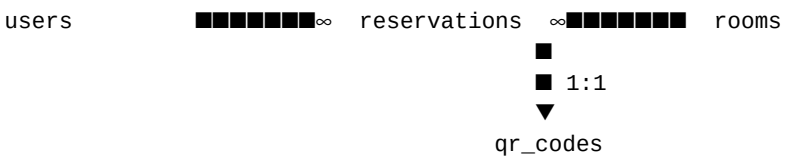
The integration also establishes the data contract on which the frontend client relies. Every JSON payload exchanged between the browser and the API ultimately resolves to a row, a join, or a constraint in the schema described below. Getting this mapping right was therefore a precondition for the rest of the system.

3. Database Design

3.1 Entity Overview

Entity	Responsibility
users	Stores authentication principals — students, staff, and admins. Email is the natural unique key;
rooms	Catalogues bookable resources (labs, classrooms, meeting rooms). Carries capacity and a deri
reservations	Time-bound bookings linking a user to a room. Holds the half-open interval used for conflict det
qr_codes	Per-room door stickers in a 1:1 relationship with a room. Used by the static QR validation flow.

3.2 Relationships



- **users** → **reservations** — one-to-many. Each user may hold many reservations; each reservation belongs to exactly one user. ON DELETE RESTRICT prevents accidental orphaning.
- **rooms** → **reservations** — one-to-many. The same protective ON DELETE RESTRICT semantics apply.
- **rooms** → **qr_codes** — one-to-one. The unique constraint on qr_codes.RoomID enforces the cardinality at the storage layer; ON DELETE CASCADE removes the QR sticker if its room is

deleted.

3.3 Canonical Schema

```
CREATE TABLE users (
    UserID         SERIAL PRIMARY KEY,
    FirstName      VARCHAR(255) NOT NULL,
    LastName       VARCHAR(255) NOT NULL,
    Email          VARCHAR(255) NOT NULL UNIQUE,
    Password       VARCHAR(255) NOT NULL,
    Role           VARCHAR(10)  NOT NULL
                   CHECK (Role IN ('Student','Admin','Staff')),
    StudentNumber  VARCHAR(50)
);

CREATE TABLE rooms (
    RoomID       SERIAL PRIMARY KEY,
    RoomName     VARCHAR(255) NOT NULL,
    RoomType     VARCHAR(100) NOT NULL,
    Capacity     INTEGER NOT NULL DEFAULT 1,
    Location     VARCHAR(255) NOT NULL,
    IsAvailable  BOOLEAN NOT NULL DEFAULT TRUE,
    QRCode       TEXT
);

CREATE TABLE reservations (
    ReservationID    SERIAL PRIMARY KEY,
    UserID           INTEGER NOT NULL REFERENCES users(UserID),
    RoomID           INTEGER NOT NULL REFERENCES rooms(RoomID),
    ReservationDate  TIMESTAMP NOT NULL,
    StartTime        TIMESTAMP NOT NULL,
    EndTime          TIMESTAMP NOT NULL,
    Status           VARCHAR(20) NOT NULL DEFAULT 'Pending',
    CreatedAt        TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    QRCodeData       TEXT
);

CREATE TABLE qr_codes (
    QRID         SERIAL PRIMARY KEY,
    RoomID       INTEGER NOT NULL UNIQUE
                 REFERENCES rooms(RoomID) ON DELETE CASCADE,
    QRCodeValue  TEXT NOT NULL,
    IsActive     BOOLEAN NOT NULL DEFAULT TRUE
);
```

3.4 Indexing Strategy

Reservation creation is the hottest write path in the system. Every attempt must be checked against existing reservations for the same room, status, and overlapping time window. To make this query logarithmic instead of a full table scan, the following composite index was added:

```
CREATE INDEX idx_reservations_conflict_check
    ON reservations (RoomID, Status, StartTime, EndTime);
```

The column order matches the WHERE-clause selectivity: filtering by *RoomID* first cuts the search space dramatically; *Status* narrows it to confirmed rows; the time bounds then perform the half-open interval check.

4. Database Integration

4.1 Provider Substitution

During prototyping, the backend ran against EF Core's *UseInMemoryDatabase* provider, which is convenient but loses every row when the process restarts. The first integration step was to swap this for the Npgsql PostgreSQL provider:

```
// Startup.cs
services.AddDbContext<AppDbContext>(options =>
    options.UseNpgsql(
        Configuration.GetConnectionString("DefaultConnection")));
```

The connection string itself was centralised in *appsettings.json* so different environments can override it without code changes:

```
"DefaultConnection":
  "Host=localhost;Database=RoomReservationDB;
   Username=postgres;Password=postgres;Port=5432"
```

4.2 Schema-to-Code Mapping

PostgreSQL folds unquoted identifiers to lowercase but preserves the case of quoted ones. Because EF Core defaults to PascalCase property names while the schema uses snake_case table names, an explicit *ToTable(...)* mapping was required to avoid runtime errors of the form *relation "Users" does not exist*:

```
protected override void OnModelCreating(ModelBuilder mb)
{
    mb.Entity<User>().ToTable("users");
    mb.Entity<Room>().ToTable("rooms");
    mb.Entity<Reservation>().ToTable("reservations");
    mb.Entity<QR>().ToTable("qr_codes");
}
```

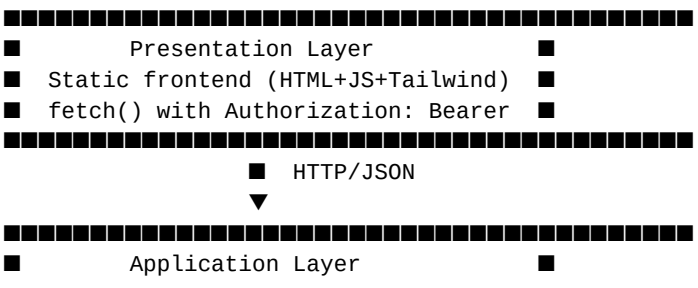
4.3 Seeding

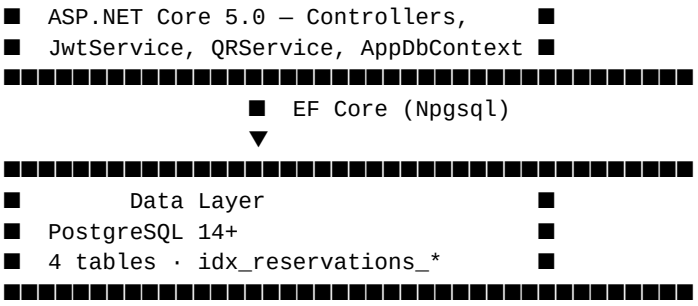
A companion *seed.sql* file provides reproducible test data — four users (Student, Admin, Staff, Student), three rooms, and three QR codes. Every insert is guarded with *ON CONFLICT (...) DO NOTHING* so the script remains idempotent across re-runs and CI pipelines.

5. Module Integration

5.1 Architecture

The integrated system follows a classical three-tier architecture consisting of a presentation tier (the static HTML/JS frontend), an application tier (the ASP.NET Core Web API), and a data tier (PostgreSQL). The boundaries between tiers are crossed only via well-defined contracts: HTTP/JSON between presentation and application, and parameterised SQL via Npgsql between application and data.





5.2 Data Flow

A representative end-to-end flow — creating a reservation — exercises every tier and is therefore the canonical integration test:

- **1.** The browser submits a JSON body to *POST /api/reservation/create* with the user's JWT in the Authorization header.
- **2.** The controller deserialises the body, reads the caller's identity from the JWT claim *NameIdentifier*, and rejects mismatched user IDs.
- **3.** EF Core issues a SELECT against *reservations* using the composite index to detect overlaps.
- **4.** If the slot is free, a row is inserted, the QRService renders a base64 PNG referencing the new reservation, and the row is updated with the JSON payload.
- **5.** The response payload (*reservationID*, *qrPayload*, *qrImage*) is JSON-serialised back to the browser, which displays the QR in a modal.

5.3 Authentication Plumbing

All non-public endpoints are protected by JWT bearer authentication. On *POST /api/auth/login* the controller queries the database for the (Email, Password) pair and, on success, asks *JwtService* to mint an HMAC-SHA256-signed token containing *NameIdentifier*, *Email*, and *Role* claims. The frontend stores this token in *localStorage* and attaches it to every subsequent request via the central *Api* wrapper.

5.4 QR Round-Trip

The QR layer interacts with the database in two distinct ways:

- **Generation.** When a reservation is confirmed, *QRService.GenerateReservationQR* serialises the reservation context to JSON and stores that payload in *reservations.QRCodeData*. The base64 PNG returned to the client is regenerated on demand from the stored payload.
- **Validation.** Two endpoints serve the two scanning scenarios. *GET /api/qr/validate* looks up a static room sticker in *qr_codes* and confirms the caller has a current confirmed reservation for that room. *POST /api/qr/validate-reservation* instead parses the JSON payload from a student's screen and verifies the reservation status and validity window directly.

6. Challenges Encountered

Several non-trivial issues surfaced during integration. They are summarised below alongside the solutions adopted.

Challenge	Description
Identifier-case mismatch	PostgreSQL preserves the case of quoted identifiers but folds unquoted ones to lowercase. The
Hard-coded build artefact	The original .csproj contained a Content reference to an absolute path on the original author's m
Asynchronous race condition	Two concurrent reservation attempts on the same time slot could each pass the overlap check b
JWT secret exposure	The signing key was initially committed to appsettings.json. Moving it to environment variables (
HTTPS redirection friction	The default app.UseHttpsRedirection() in development forced the frontend to chase a self-signe
Capacity semantics drift	The early controller decremented rooms.Capacity on every booking, conflating physical capacity

7. Solutions Applied

• **Schema standardisation.** All table names were normalised to snake_case (users, rooms, reservations, qr_codes), and EF Core mappings were made explicit through ToTable(...). Future column renames are required to be performed in both the C# property and the SQL schema in the same commit.

• **Composite index.** idx_reservations_conflict_check was introduced on (RoomID, Status, StartTime, EndTime) to keep the overlap query logarithmic regardless of the table's growth.

• **Idempotent seed.** seed.sql uses ON CONFLICT (...) DO NOTHING for every insert, so re-running the script never produces duplicates and never aborts CI.

• **Centralised connection.** appsettings.json holds a single DefaultConnection string; environment-specific overrides go in appsettings.Development.json or environment variables.

• **Build hygiene.** The hard-coded Content reference in the .csproj was removed, .vs/ and bin/obj/ were added to .gitignore, and a clean integrated repository structure was published with explicit READMEs in each module folder.

• **Demo-friendly defaults.** HTTPS redirection is suppressed in Development to avoid self-signed-cert issues during local demos; CORS is opened to AllowAll for the same reason. Both are reverted in production.

8. Final Architecture

The result of the integration work is a single coherent repository in which the three modules can be brought up independently and composed without bespoke configuration. A new contributor can clone the repository, run a single SQL bootstrap, start the API, serve the frontend statically, and have a working system in under five minutes.

```
RoomReservationSystem-Integrated/
■■■ backend/   ASP.NET Core Web API
■■■ frontend/  Static HTML + Tailwind + Vanilla JS
■■■ database/  schema.sql + seed.sql
■■■ docs/      Reports 1–5
■■■ .gitignore
■■■ .env.example
■■■ README.md
```

The clean separation buys three concrete properties:

• **Replaceability.** Any tier can be swapped without touching the other two as long as the contract is honoured. The frontend could be rewritten in React, the backend re-implemented in Node, or the database migrated to MySQL — each in isolation.

• **Testability.** The integration tests can target the HTTP boundary directly, with the database seeded by seed.sql for deterministic fixtures.

• **Deployability.** The same repository layout supports local development, CI, and a future Docker Compose configuration without restructuring.

9. Summary

Deliverable	Status
PostgreSQL schema	Complete (4 tables, FKs, CHECK, composite index)
Idempotent seed data	Complete (4 users, 3 rooms, 3 QR codes)
EF Core provider switch	Complete (InMemory → Npgsql)
Schema-to-code mapping	Complete (explicit ToTable for all entities)
JWT authentication wiring	Complete (HMAC-SHA256, role claims)
Module integration repository	Complete (backend + frontend + database)
Documentation	Complete (this report + per-module READMEs)

With these pieces in place, the system meets the project's data and integration objectives. The remaining items on the project backlog — registration endpoint, password hashing, EF Core migrations, and automated tests for the C# layer — are independent extensions that can be developed without renegotiating the contracts established here.